Smart prediction for refactorings in the software test code

Luana Martins
Federal University of Bahia (UFBA)
Salvador, Bahia, BRA
martins.luana@ufba.br

Heitor Costa
Federal University of Lavras (UFLA)
Lavras, Minas Gerais, BRA
heitor@ufla.br

Carla Bezerra
Federal University of Ceara (UFC)
Quixadá, Ceara, BRA
carlailane@ufc.br

Ivan Machado
Federal University of Bahia (UFBA)
Salvador, Bahia, BRA
ivan.machado@ufba.br

ABSTRACT

Test smells are bad practices to either design or implement a test code. Their presence may reduce the test code quality, harming the software testing activities, primarily from a maintenance perspective. Therefore, defining strategies and tools to handle test smells and improve the test code quality is necessary. State-of-the-art strategies encompass automated support mainly based on hard thresholds of rules, static and dynamic metrics to identify the test smells. Such thresholds are subjective to interpretation and may not consider the complexity of the software projects. Moreover, they are limited as they do not automate test refactoring but only count on developers' expertise and intuition. In this context, a technique that uses historical implicit or tacit data to generate knowledge could assist the identification and refactoring of test smells. This study aims to establish a novel approach based on machine learning techniques to suggest developers refactoring strategies for test smells. As an expected result, we could understand the applicability of the machine learning techniques to handle test smells and a framework proposal that helps developers in decision-making regarding the refactoring of test smells.

CCS CONCEPTS

• **Software and its engineering** → *Software testing and debugging*;
• **Computing methodologies** → *Machine learning approaches.*

KEYWORDS

Test Smells, Machine Learning, Software Quality

ACM Reference Format:
Luana Martins, Heitor Costa, Carla Bezerra, and Ivan Machado. 2021. Smart prediction for refactorings in the software test code. In *Brazilian Symposium on Software Engineering (SBES '21), September 27-October 1, 2021, Joinville, Brazil.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3474624.3477070

SBES '21, September 27-October 1, 2021, Joinville, Brazil

ACM ISBN 978-1-4503-9061-3/21/09…$15.00
https://doi.org/10.1145/3474624.3477070

1 INTRODUCTION

Software testing is a fundamental activity for software quality assurance. That activity comprises developing test cases helpful in finding bugs and preventing software regressions [11, 19]. However, software companies often do not prioritize software testing activities because they face difficulties selecting qualified personnel and allocating resources for testing activities [18]. Additionally, the testing activities consume much time and effort, especially when done manually. The effort to perform testing activities depends on the artifacts under test, e.g., a large and complex production class requires more effort to perform tests [27].

Developing tests for large and complex production classes are not trivial, leading developers to use bad practices to design or implement the test code [11, 26]. A particular category of bad practices in the test code is known as test smells. Their presence can negatively affect the test code quality, harming the software testing activities, primarily from a maintenance perspective [5, 19, 25]. Several studies in the literature have proposed strategies with automated support to handle test smells and improve test code quality. However, such strategies rely on hard thresholds of rules, static and dynamic metrics to detect the test smells [5, 19–21, 23, 25, 28].

Spadini et al. [25] pointed out that the current strategies are simplistic and can impair the developers of perceiving the test smells as problematic. The threshold values are subjective to interpretation, depending on the developers' knowledge about the software complexity and test smells. Additionally, the strategies are limited as they do not automate test refactoring but only count on developers' expertise and intuition. These limitations could harm patterns discovering and insights that support the decision-making to handle test smells. Therefore, it is essential to investigate other methods, techniques, and tools to prevent, detect, and remove test smells from the test code.

This study proposes a novel approach that uses historical implicit or tacit data to generate knowledge about test smells. We presented an initial framework proposal based on machine learning techniques to identify test smells and suggest refactoring strategies on test code to remove them. In this preliminary study, we investigated the performance of seven machine learning algorithms to detect test smells in terms of accuracy, precision, recall, and f1-score. Those algorithms presented an excellent overall accuracy. However, improvements should be made to increase the accuracy. Furthermore, we propose a set of research questions to drive future investigations.

2 BACKGROUND

2.1 Test smells

Test smells are anti-patterns adopted to implement or design the test code [19]. The insertion of test smells in the test code is due to the complexity of the software artifacts under test [26]. Furthermore, the presence of the test smells can affect the test code quality, mainly its comprehension and readability [5, 25]. Consequently, test smells can reduce the effectiveness of the test cases to detect faults and the ability for developers to interact with them [5, 19]. Several studies have investigated anti-patterns in the test code, and their effects [5, 11, 23, 25, 28, 29]. The literature reports more than 139 test smells [11], but the available tools support only a subset of test smells [21]. Initially, in our study, we analyzed 4 test smells, which were validated through a survey with developers that either own or contribute to open source systems that exhibited those smells [21]:

- **Assertion Roulette (AR).** A test method that contains assertion statements without an explanation message. Non-documented assertions impact the test code readability and understandability;
- **Conditional Test Logic (CTL).** A test method that contains conditional and loop statements. Conditions within the test method can modify the test behavior and its expected output;
- **Eager Test (ET).** A test method calls multiple methods of the production class. The test method has several responsibilities, making it difficult to comprehend and maintain;
- **Verbose Test (VT).** A test method with more than 30 lines. It may indicate that the method has several responsibilities, impacting the test method maintenance.

2.2 Structural metrics

Software metrics provide quantitative evidence on software quality characteristics (*e.g.*, reusability, maintainability, and testability) [13]. Software engineers can use them to understand the code structure. Such evidence supports the software planning, creating, and evaluation processes [27]. The literature proposes software metrics to different development contexts. In this study, we focused on object-oriented metrics to assess Java open-source projects. Table 1 lists our set of metrics [3, 9].

2.3 Machine learning algorithms

There are various machine learning (ML) algorithms that we can categorize as supervised, unsupervised, semi-supervised, and reinforcement learning [14, 24]. *Supervised learning* is typically the task of ML to learn a function that maps an input to an output based on sample input-output pairs [12]. *Unsupervised learning* analyzes unlabeled datasets without the need for human interference, i.e., a data-driven process [12]. *Semi-supervised learning* can be defined as a hybridization of supervised and unsupervised methods, as it operates on both labeled and unlabeled data [24]. Finally, *Reinforcement learning* enables software agents and machines to automatically evaluate the optimal behavior in a particular context or environment to improve its efficiency [14].

Initially, in our study, we focused on supervised learning to verify the feasibility of using ML techniques to test smells detection. We used seven algorithms, grouped into five supervised ML techniques:

- **Decision Trees (DT)**. DT approximates robust discrete functions in the presence of noise and is capable of expressing

Table 1: Software metrics description according to Aniche et al. [3] and Chidamber and Kemerer [9].

Acronym	Name	Description
assignmentsQty	Quantity of assignments	Counts the number of assignments in a class or method
CBO	Coupling Between Objects	Counts the number of afferent and efferent couplings of a class or method
DIT	Depth Inheritance Tree	Counts the number of parents for a class
LOC	Lines of Code	Counts the number of lines of a class or method
LCOM	Lack of Cohesion of Methods	Counts the number of method pairs not using common variables
loopQty	Quantity of loops	Counts the number of loop structures in a class or method
maxNestedBlocksQty	Max nested blocks	Counts the number of conditional statements in a class or method
methInvQty	Quantity of directly invocations	Counts the number of directly invocations of a method
methInvLocalQty	Quantity of local invocations	Counts the number of local invocations of a method
methInvIndirectQty	Quantity of indirect local invocations	Counts the number of local indirectly invocations of a method
NOF	Number of Fields	Counts the number of fields (all modifiers) in a class
NOM	Number of Methods	Counts the number of methods (all modifiers) in a class
NPRIF	Number of PRIvate Fields	Counts the number of private fields of a class
NPRIM	Number of PRIvate Methods	Counts the number of private methods of a class
NPROM	Number of PROtected Methods	Counts the number of protected methods of a class
numbersQty	Quantity of numeric literals	Counts the number of numeric literals in a class or method
RFC	Response for a Class	Counts the number of methods that respond to class or method
returnQty	Quantity of returns	Counts the number of return statements in a class or method
stringQty	Quantity of String literals	Counts the number of string literals in a class or method
tryCatchQty	Quantity of try/catches	Counts the number of try/catches blocks in a class or method
uniqueWordsQty	Quantity of unique words	Counts the number of unique words in a class or method
variablesQty	Quantity of declared variables	Counts the number of variables declared in a class or method
WMC	Weight Method Class	Counts the number of paths of all methods in a class or method

disjunctive learning. ID3 is an algorithm that classifies the instances by sorting them in the decision and leaves nodes of a tree [22];

- **Instance-based Learning (IBL)**. IBL relies on statistically based and lazy-learning algorithms. Examples of IBL algorithms include K-Nearest Neighbors (KNN) and Distance-Weighted Nearest Neighbors (DWNN) that calculate the (weighted) distance between the unobserved and observed instances[1];
- **Logistic Regression (LR)**. LR uses linear functions to establish a correlation between the unobserved and observed instances. Examples of LR algorithms include one-vs-rest (LRO) and multinomial logistic regression (LRM) [6];
- **Random Forest (RF)**. RF contains trees based on the values of a random subset of instances experimented individually and with the same distribution for all trees in the forest [7];
- **Support Vector Machines (SVM)**. SVM computes a hyperplane in high-dimensional space to classify data into predefined classes. The algorithm searches for the best hyperplane for error minimization and geometric margin maximization. SVM usually does not work well for highly imbalanced data [10].

3 FRAMEWORK

3.1 Solution proposal

Figure 1 shows a high-level overview of the framework proposal that encompasses seven modules.

Test Code Quality Planning, Monitoring, and Control. This module uses information from the *Software Measurement* and *Knowledge Management* modules to plan, monitor, and control the software testing activities. Besides, it provides management information and the test quality goals useful in the *Context Identification* and *Smart Prediction* modules. As an output, the software engineer can check the quality goals against the measurement and knowledge information to plan the allocation of resources and prioritize the development and maintenance of testing artifacts that require more effort to achieve the expected quality level.

Context Identification. This module uses the management information from the *Test Code Quality Planning, Monitoring, and Control* to understand the software context, identify the test smells, and suggest refactoring strategies for test smells in the given context. The context splits into *(i) Project Level* that uses the *Software Measurement* data to reflect the project properties (size, complexity, and maturity), and *(ii) Environment Level* that uses the *Knowledge Management* to reflect the subjective factors influencing the adoption of refactoring strategies (e.g., business domain, expertise, and culture). This module can group the software projects into clusters representing different contexts to investigate whether the detection of test smells and refactoring strategies generalize to different contexts.

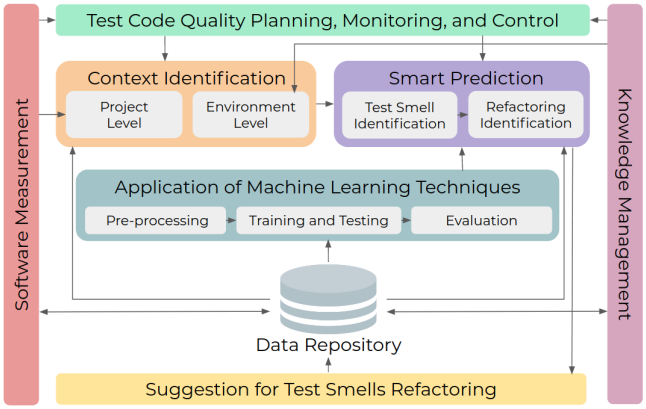

Figure 1: Overview of the framework proposal.

Application of Machine Learning Techniques. This module provides ML algorithms for the *Smart Prediction* module. For the application of the algorithms, the module encompasses three phases: (i) *Pre-Processing* analyzes the data stored in the Data Repository to threaten the outliers and remove unnecessary features (software metrics); (ii) *training and testing* uses one technique to split the dataset into training and testing data. It serves to evaluate the accuracy of the algorithms in each fold; and (iii) *evaluation* compares the mean precision, recall, accuracy, and f1-score among the different models after the validation.

Smart Prediction. This module identifies the goals established for the software project in the *Test Code Quality Planning, Monitoring and Control* module and detects project context based on the output of the *Context Identification* module. It also runs the *Application of Machine Learning Techniques* module for the test smells identification. For each instance returned, it runs the same module to identify the refactoring strategies useful for the context.

Suggestion for Test Smells Refactoring. This module provides the developers a list of test smells instances and possible refactoring strategies. The developers should analyze whether and which refactoring solution applies in the given development context. Therefore, the framework captures the context and feeds it back to improve test smells detection and suggest refactoring strategies.

Software Measurement. This module implements automated support to calculate metrics from the test and production code necessary to provide useful information for identifying the test smells. It stores all data collected from the software projects in the data repository.

Knowledge Management. This module uses the knowledge extracted from the user and the Data Repository by improving the test code quality. It also captures, evaluates, and makes available new knowledge produced when executing the modules. Finally, it stores all data collected in the data repository.

3.2 Research questions

We aim to propose a framework to detect test smells and predict refactoring strategies based on the software context and ML techniques. We plan to advance on the research and add the produced results to one framework, turning it into a more robust approach to support. We derived four Research Questions (RQs):

RQ$_1$: *How accurate are machine learning algorithms in detecting test smells?*

RQ$_2$: *How accurate are machine learning algorithms in predicting refactoring for test smells?*

RQ$_3$: *Are there predictive models carried over to different contexts to detect test smells?*

RQ$_4$: *Are there predictive models carried over to different contexts to predict refactoring for test smells?*

ML algorithms perform differently depending on the task. Each RQ explores supervised ML algorithms in terms of precision, accuracy, recall, and f1-score. To answer RQ$_1$, we intend to mine the GitHub repository to select Java open-source projects. We plan to use automated support to calculate the software metrics and detect the test smells instances, which we can validate manually. To answer RQ$_2$, we plan to mine the GitHub repository for the selected projects and collect data regarding the refactoring strategies applied to remove test smells from the test code. We will store the data regarding the test smells detection and refactoring strategies into a data repository to input ML algorithms. In addition to comparing the accuracy of the ML algorithms, we could also compare them with the automated tools. To answer RQ$_3$ and RQ$_4$, we intend to cluster the projects into groups to analyze whether the ML algorithms generalize to other contexts. We can create clusters to delimit the contexts considering the projects' complexity, size, and business domain.

4 PRELIMINARY RESULTS

Mining the GitHub repository is a time-consuming activity because it has more than 8 million projects. Therefore, we used the 147,991 projects listed by Loriot et al. [17] to mine the GitHub and find projects that meet the following criteria: (i) non-forked projects, (ii) projects using Java as the primary programming language, and (iii) projects with tests written with the JUnit framework. This preliminary study presents the results from 57 popular projects (i.e., projects sorted using two indexes: number of stars and forks).

Our initial objective was to investigate the feasibility of using ML techniques for test smell detection. We designed the detection of test smells as a binary classification problem, which allowed us to check the output of the ML algorithms against the manual classification of labeled instances extracted from 57 software projects. We selected seven supervised learning algorithms that support classification problems: DT, DWNN, KNN, LRM, LRO, RF, and SVM.

We used the `JNose Test` tool[1] to collect four test smells: AR, CTL, ET, and VT. That tool returns the test smells instances in different granularities (line, block, method, and method, respectively) [28]. To address the test smells detection as a classification problem, we converted the number of test smells in the methods to 'true' or 'false'. While converting the values, we manually checked whether the results returned by `JNose Test` are not false positives. Besides, we limited the dataset to 500 instances labeled with a specific test smell ('AR', 'CTL', 'ET', 'VT'), or no test smell at all ('None').

We also used the `CK Metrics` tool[2] to collect the software metrics at the method and class levels. As the method level is common for detecting test smells and calculating software metrics, we established a link between them at the method level (Figure 1 - *Software Measurement* module). The dataset has 23 features corresponding to the software metrics (Figure 1 - Data Repository).

[1]Available at: https://github.com/arieslab/jnose
[2]Available at: https://github.com/mauricioaniche/ck

Table 2: Descriptive statistics for the initial dataset containing 500 instances extracted from 57 software projects.

Metric	Mean	SD	Min	Q1	Q2	Q3	Max
CBO	4.6	3.3	1.0	3.0	4.0	5.0	21.0
WMC	1.7	1.4	1.0	1.0	1.0	2.0	13.0
RFC	7.8	7.3	0.0	3.0	6.0	10.0	55.0
LOC	15.0	13.5	2.0	6.0	9.0	21.0	68.0
variablesQty	3.5	3.9	0.0	1.0	2.0	5.0	24.0
methInvQty	7.8	7.3	0.0	3.0	6.0	10.0	55.0
methInvLocalQty	0.5	0.6	0.0	0.0	0.0	1.0	3.0
methInvIndirectQty	0.5	0.8	0.0	0.0	0.0	1.0	6.0
stringLiteralsQty	4.8	7.2	0.0	0.0	2.0	6.0	44.0
numbersQty	2.8	5.4	0.0	0.0	1.0	3.0	67.0
assignmentsQty	4.1	4.6	0.0	1.0	2.0	6.0	26.0
maxNestedBlocksQty	0.5	0.8	0.0	0.0	0.0	1.0	5.0
uniqueWordsQty	21.1	11.3	2.0	13.0	20.0	27.0	67.0

Table 2 presents the descriptive statistics of the initial dataset containing the software metrics. For example, the LOC metric shows that the number of lines of a test method ranged from 2 (*Min*) to 68 lines (*Max*). The *Q1* represents $\frac{1}{4}$ of the dataset, and it is composed only by test methods until six lines. The *Q3* represents $\frac{3}{4}$ of the dataset, and it is composed of test methods that contain between 10 and 55 lines. The analyzed test methods presented nine lines on average (*Q2*), with a mean *Mean* value of 15 lines, and a standard deviation *SD* of 13.5, indicating high dispersion of values.

Based on the descriptive statistics, we started classifying test smells instances (Figure 1 - *Application of Machine Learning Techniques* module). In the pre-processing phase, we removed the metrics with values equal to zero for all instances because they are class-level metrics (e.g., LCOM, DIT, NOF, NOM, NPRIF, NPRIM, and NPROM) or do not match the characteristics of the selected method (e.g., assignmentsQty, loopQty, and returnQty). Additionally, we used the imputation method to deal with the outliers. For example, the LOC metrics present a Q3 of 21 LOC; however, the Max value is 68 LOC. Therefore, we substituted the outlier for a value in Q3, as it continues representing a long method. Additionally, we normalized the values to the same range to avoid the ML algorithms to learn with a scale of the data. After the pre-processing phase, 13 features remained. Next, we performed the training and test phase. We used the 10-fold cross-validation to split the data into training and testing data, and we also applied the parameters tuning for the ML algorithms: DT, DWNN, KNN, LRM, LRO, RF, and SVM. In the evaluation phase, we calculated the accuracy, precision, recall, and f1-score using the mean of the 10-fold execution.

Table 3 presents the evaluation of each algorithm grouped by the dataset labels. The algorithms present high precision and recall scores when classifying ET (f1-score > 90%) and CTL (f1-score > 80%) instances. On the other hand, the algorithms have less precision and recall scores to classify instances with AR, VT, or no test smells. Mostly, the algorithms classify instances with no test smell and VT as an AR. Also, different algorithms perform better to detect a specific test smell, e.g., the RF algorithm is good to classify AR, and no test smells instances. The DWNN and DT algorithms are suitable to classify ET, CTL, and VT instances.

Table 3: Scores for the ML algorithms to classify the test smells instances.

Labels	Algorithms	Accuracy	Precision	Recall	F1-Score
	DT	93.7	84.3	85.2	84.7
	DWNN	93.2	82.4	85.2	83.8
	KNN	94.1	46.5	53.1	49.6
No	LRM	92.7	85.4	79.5	82.4
	LRO	92.4	83.5	80.7	82.1
	RF	95.3	88.6	88.6	88.6
	SVM	97.3	87.8	81.8	84.7
	DT	92.2	80.4	82.2	81.3
	DWNN	93.0	81.9	85.6	83.7
	KNN	90.5	73.1	87.8	79.8
AR	LRM	93.4	83.2	87.8	85.4
	LRO	92.2	83.7	80.0	81.8
	RF	94.2	86.5	85.6	86.0
	SVM	92.8	81.9	85.6	83.7
	DT	99.0	97.8	97.8	97.8
	DWNN	99.5	97.8	100.0	98.9
	KNN	96.2	90.3	93.3	91.8
ET	LRM	95.5	87.5	93.3	90.3
	LRO	95.9	89.4	93.3	91.3
	RF	99.0	95.7	100.0	97.8
	SVM	96.0	91.1	91.1	91.1
	DT	97.6	94.4	94.4	94.4
	DWNN	97.1	91.5	95.6	93.5
	KNN	94.1	85.1	88.9	87.0
CTL	LRM	94.1	85.1	88.9	87.0
	LRO	93.3	81.2	91.1	85.9
	RF	96.4	90.3	93.3	91.8
	SVM	94.4	85.3	90.0	87.6
	DT	94.1	87.6	84.8	86.2
	DWNN	93.2	89.9	77.2	83.0
	KNN	96.2	89.0	70.7	78.8
VT	LRM	91.6	84.3	76.1	80.0
	LRO	90.9	82.1	75.0	78.4
	RF	94.0	88.4	82.6	85.4
	SVM	93.5	86.5	83.7	85.1

Figure 2 shows the mean of the metrics we used to evaluate the algorithms. The RF outperforms the other algorithms, presenting the mean values for all metrics greater than 90%. In contrast, the linear classifiers LRO and LRM underperform the other algorithms, presenting the mean values for all metrics less than 84%. SVM is also a linear classifier, which presents mean values slightly better than LRO and LRM when using the kernel trick. It emphasizes that our data is not linearly separable. Considering the non-linear classifiers, the lazy ones, KNN and DWNN, are more sensitive to the local values. Due to the sparsity of our dataset, DT and RF classifiers perform well compared with the lazy algorithms.

5 RELATED WORK

Aljedaani et al. [2] mapped twenty-two state-of-the-art tools useful to detect test smells, from which only five support refactoring strategies. Although the tools support a common subset of test smells, there is no consensus on detecting and refactoring them. Each tool implements a strategy to handle test smells, grouping mainly them into: *(i) metrics-based*, which calculates structural and dynamic metrics and the interpretation of their values in front of predefined thresholds indicate the presence of test smells (TeReVis and TeReDetect [15]); *(ii) rules-based*, which identifies and matches

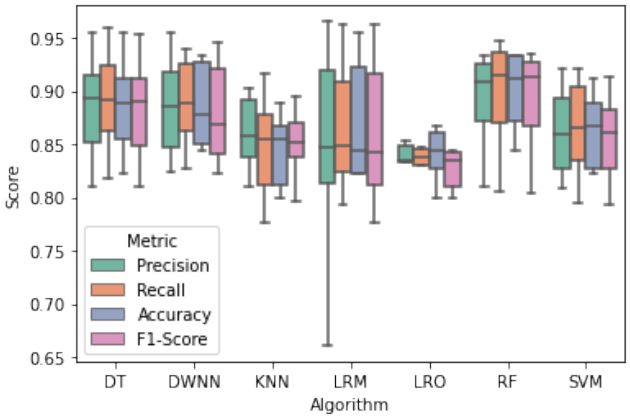

Figure 2: Boxplot for the executions of the k-fold cross-validation using the ML algorithms.

the code patterns with predefined thresholds to indicate the presence of test smells (e.g., JNose Test [28], tsDetect [21], and RAIDE[3] [23]; *(iii) information-retrieval*, which extracts textual components from the test code to apply the detection rules (e.g., DARTS[3] [16] and TASTE [20]). These tools often rely on hard thresholds to detect test smells, particularly the metrics and rules-based tools.

For example, the tsDetects considers all test methods with more than one assertion without an explanation as a test smell. Spadini et al. [25] pointed out that such detection strategies are simplistic and may impair the developers of perceiving the test smells as problematic. Besides that, the developers can perceive an unsettling test smell as an acceptable trade-off when considering the domain context (e.g., the software complexity, environment, application type). Several studies point to the need for further research using ML techniques to predict test smells. Still, there is no study exploring these techniques to overcome the current limitations.

We can use ML techniques to detect, predict and recommend refactoring strategies to design and implement flaws in other software artifacts. Similar to test smells, the bad practices to design or implement the source code are known as code smells. In this context, Azeem et al. [4] reviewed the literature to provide an overview of ML techniques to predict code smells. They analyzed 15 studies and provided a list with the most common code smells and ML algorithms used to predict them. As a result, they noticed that the absence of metrics analysis could represent an opportunity to reduce the subjectivity in the interpretation of code smells.

Similarly, Caram et al. [8] analyzed 26 studies to identify how ML algorithms support code smell detection. Results indicate that the techniques perform close to each other; DT, RF, and KNN had the best performance overall, and the NB algorithm had the worst performance. These results converge to our preliminary results, which indicates that the usage of ML algorithms is also promising for test smells detection. Lastly, Aniche et al. [3] presented a practical application of ML techniques to identify and predict refactoring strategies to code smells. They analyzed software classes and methods to collect historical data of refactoring operations in practice

[3]Examples of tools that provide support for refactoring test smells.

and trained the ML algorithms to predict refactoring operations to a given code. Results indicate that the ML models generalize well in different contexts. More specifically, they consider as the context projects implementing libraries, frameworks, and mobile apps. Therefore, we hypothesize that the ML models could fit better to a specific context when handling test smells.

6 FINAL REMARKS

This paper presents a high-level overview of a framework proposal to provide developers with refactoring strategies for test smells. This framework plans to collect historical implicit or tacit data from the software projects and feed ML algorithms. The use of a large-scale dataset makes the ML algorithms more susceptible to reach generalizable findings. Therefore, we intend to establish a novel approach that considers the development context to suggest more assertive information to developers, increasing their productivity and effectiveness when handling test smells.

The framework proposal serves as a starting point for future research. We identified some RQs to drive further investigations that will provide us results to improve the framework. More specifically, we intend to detail the framework processes to gather and analyze the data from software projects to generate knowledge. As a preliminary work, we presented the GitHub repository mining to select Java projects and the overall idea of collecting software metrics to detect test smell. We trained seven ML algorithms to investigate the feasibility of detecting test smells. As a result, we could identify that the algorithms present a good performance for test smells detection. However, a further investigation is required to identify whether the good performance is due to: i) the projects' characteristics, e.g., we can group the projects by size to investigate whether there is a difference between those groups; and ii) the selection of instances that compose our dataset, e.g., a dataset containing more than one test smell per instance depicts a more realistic scenario of software testing code.

ACKNOWLEDGMENTS

This research was partially funded by INES 2.0; CNPq grants 465614/2014-0 and 408356/2018-9 and FAPESB grants JCB0060/2016 and BOL0188/2020.

REFERENCES

[1] David W Aha, Dennis Kibler, and Marc K Albert. 1991. Instance-based learning algorithms. *Machine learning* 6, 1 (1991), 37–66.
[2] Wajdi Aljedaani, Anthony Peruma, Ahmed Aljohani, Mazen Alotaibi, Mohamed Wiem Mkaouer, Ali Ouni, Christian D. Newman, Abdullatif Ghallab, and Stephanie Ludi. 2021. Test Smell Detection Tools: A Systematic Mapping Study. arXiv:2104.14640 [cs.SE]
[3] Maurício Aniche, Erick Maziero, Rafael Durelli, and Vinicius Durelli. 2020. The Effectiveness of Supervised Machine Learning Algorithms in Predicting Software Refactoring. arXiv:2001.03338 [cs.SE]
[4] Muhammad Ilyas Azeem, Fabio Palomba, Lin Shi, and Qing Wang. 2019. Machine learning techniques for code smell detection: A systematic literature review and meta-analysis. *Information and Software Technology* 108 (2019), 115–138.
[5] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and Dave Binkley. 2015. Are test smells really harmful? an empirical study. *Empirical Software Engineering* 20, 4 (2015), 1052–1094.
[6] Christopher M. Bishop. 2006. *Pattern Recognition and Machine Learning (Information Science and Statistics).* Springer-Verlag, Berlin, Heidelberg.
[7] Leo Breiman. 2001. Random Forests. *Mach. Learn.* 45, 1 (Oct. 2001), 5–32.
[8] Frederico Luiz Caram, Bruno Rafael De Oliveira Rodrigues, Amadeu Silveira Campanelli, and Fernando Silva Parreiras. 2019. Machine learning techniques for code smells detection: a systematic mapping study. *International Journal of Software Engineering and Knowledge Engineering* 29, 02 (2019), 285–316.
[9] S.R. Chidamber and C.F. Kemerer. 1994. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering* 20, 6 (1994), 476–493.
[10] Corinna Cortes and Vladimir Vapnik. 1995. Support-Vector Networks. *Mach. Learn.* 20, 3 (Sept. 1995), 273–297.
[11] Vahid Garousi and Barış Küçük. 2018. Smells in software test code: A survey of knowledge in industry and academia. *Journal of Systems and Software* 138 (2018), 52–81.
[12] Jiawei Han, Micheline Kamber, and Jian Pei. 2011. Data mining concepts and techniques third edition. *The Morgan Kaufmann Series in Data Management Systems* 5, 4 (2011), 83–124.
[13] ISO/IEC 25010. 2011. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.* ISO - ISO/IEC 25010:2011.
[14] Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. 1996. Reinforcement learning: A survey. *Journal of artificial intelligence research* 4 (1996), 237–285.
[15] Negar Koochakzadeh and Vahid Garousi. 2010. TeCReVis: A Tool for Test Coverage and Test Redundancy Visualization. In *Testing – Practice and Research Techniques*, Leonardo Bottaci and Gordon Fraser (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 129–136.
[16] Stefano Lambiase, Andrea Cupito, Fabiano Pecorelli, Andrea De Lucia, and Fabio Palomba. 2020. Just-In-Time Test Smell Detection and Refactoring: The DARTS Project. In *Proceedings of the 28th International Conference on Program Comprehension* (Seoul, Republic of Korea). ACM, New York, NY, USA, 441–445.
[17] Benjamin Loriot, Fernanda Madeiral, and Martin Monperrus. 2020. Styler: Learning Formatting Conventions to Repair Checkstyle Errors. arXiv:1904.01754 [cs.SE]
[18] Silvana Melo, Veronica Moreira, Leo Natan Paschoal, and Simone Souza. 2020. Testing Education: A Survey on a Global Scale. In *In 34th Brazilian Symposium on Software Engineering* (Natal, Brazil). ACM, New York, NY, USA, 554–563.
[19] Fabio Palomba, Dario Di Nucci, Annibale Panichella, Rocco Oliveto, and Andrea De Lucia. 2016. On the Diffusion of Test Smells in Automatically Generated Test Code: An Empirical Study. In *Proceedings of the 9th International Workshop on Search-Based Software Testing* (Austin, Texas). ACM, New York, NY, USA, 5–14.
[20] Fabio Palomba, Andy Zaidman, and Andrea De Lucia. 2018. Automatic Test Smell Detection Using Information Retrieval Techniques. In *2018 IEEE International Conference on Software Maintenance and Evolution.* IEEE, New York, Ny, USA, 311–322.
[21] Anthony Peruma, Khalid Almalki, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2020. TsDetect: An Open Source Test Smells Detection Tool. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA). ACM, New York, NY, USA, 1650–1654.
[22] J. Ross Quinlan. 1993. *C4.5: Programs for Machine Learning.* Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
[23] Railana Santana, Luana Martins, Larissa Rocha, Tássio Virgínio, Adriana Cruz, Heitor Costa, and Ivan Machado. 2020. RAIDE: A Tool for Assertion Roulette and Duplicate Assert Identification and Refactoring. In *Proceedings of the 34th Brazilian Symposium on Software Engineering* (Natal, Brazil). ACM, New York, NY, USA, 374–379.
[24] Iqbal H Sarker, ASM Kayes, Shahriar Badsha, Hamed Alqahtani, Paul Watters, and Alex Ng. 2020. Cybersecurity data science: an overview from machine learning perspective. *Journal of Big Data* 7, 1 (2020), 1–29.
[25] Davide Spadini, Martin Schvarcbacher, Ana-Maria Oprescu, Magiel Bruntink, and Alberto Bacchelli. 2020. Investigating Severity Thresholds for Test Smells. In *Proceedings of the 17th International Conference on Mining Software Repositories* (Seoul, Republic of Korea). ACM, New York, NY, USA, 311–321.
[26] Amjed Tahir, Steve Counsell, and Stephen G MacDonell. 2016. An empirical study into the relationship between class features and test smells. In *In 23rd Asia-Pacific Software Engineering Conference.* IEEE, New York, Ny, USA, 137–144.
[27] Valerio Terragni, Pasquale Salza, and Mauro Pezzè. 2020. Measuring Software Testability Modulo Test Quality. In *Proceedings of the 28th International Conference on Program Comprehension* (Seoul, Republic of Korea). ACM, New York, NY, USA, 241–251.
[28] Tássio Virgínio, Luana Martins, Larissa Rocha, Railana Santana, Adriana Cruz, Heitor Costa, and Ivan Machado. 2020. JNose: Java Test Smell Detector. In *Proceedings of the 34th Brazilian Symposium on Software Engineering* (Natal, Brazil). ACM, New York, NY, USA, 564–569.
[29] Tássio Virgínio, Luana Almeida Martins, Larissa Rocha Soares, Railana Santana, Heitor Costa, and Ivan Machado. 2020. An Empirical Study of Automatically-Generated Tests from the Perspective of Test Smells. In *Proceedings of the 34th Brazilian Symposium on Software Engineering* (Natal, Brazil). ACM, New York, NY, USA, 92–96.